Local C++ setup: VSCode / CLion

Choose a path based on whether VSCode is already configured. Both paths use C++17 and finish by running Hello World. Classes continue to use the instructor-provided CPP-Classroom.

中文 · Complete offline PDF · Hello World source

Path 1: VSCode is ready

Keep your existing VSCode settings and prepare the compiler for your operating system:

- **Windows:** Follow the reference video and the guide's MSYS2 installation and PATH configuration steps.

- **macOS:** Run `xcode-select --install` to install Xcode Command Line Tools. The `g++` command invokes Apple Clang, which supports the course's C++17 programs.

- **Linux:** Run `g++ -v` first. Use an existing compiler or install one for your distribution if needed. The guide includes Ubuntu/Debian and Fedora commands.

Completion: run `g++ -v`, then compile Hello World with `g++ -std=c++17` and run it in the VSCode integrated terminal.

VSCode Chinese guide · Chinese PDF · English guide · English PDF

Path 2: VSCode is not configured

Follow the existing **CLion setup with a Student Pack license applied for using your university email**. Complete the application, installation, and activation. Use bundled MinGW on Windows or Apple Clang from Xcode Command Line Tools on macOS.

Create a C++ Executable project, select C++17, copy the accompanying `hello.cpp` into `main.cpp`, and click Run.

Completion: confirm `Hello World` in the CLion Run window.

CLion Chinese guide · Chinese PDF · English guide · English PDF

For CLion on Linux/Ubuntu, continue using your Linux course environment. Run `sudo apt update` and `sudo apt install build-essential gdb cmake` to install the toolchain, then open this module's CMake project in CLion. See the module README for other build options.

The complete offline PDF contains this page, the VSCode guide, and the CLion guide, in that order.

Back to the C++ module

Local C++ setup in VSCode (Windows / macOS / Linux)

If VSCode is already installed and ready to use, install g++, then check its version and compile and run Hello World in the VSCode integrated terminal to complete setup. Use the C++17 standard.

If VSCode is not configured, follow the existing CLion setup: CLion Chinese guide · CLion English guide.

1. Install g++ for your operating system

This guide is for students who have already installed VSCode and can use it. Keep your existing settings and install the compiler for your operating system. **Installing VSCode or the C/C++ extension does not install g++**. If `g++ -v` already works in the VSCode terminal, go directly to the Hello World check in Section 2.

Select **Terminal → New Terminal**. Use the Terminal panel that opens below to check the compiler, compile your code, and run it. Do not enter these commands in Output or Debug Console. Use PowerShell on Windows, or zsh or bash on macOS/Linux.

Windows

Reference video: C++ environment setup on Windows. If VSCode is already installed, focus on installing the compiler and configuring PATH, then return to this guide for verification.

The following steps install g++ through MSYS2 on Windows x64:

1. Download and install MSYS2 from the MSYS2 website. These steps assume the default installation directory, `C:\msys64`.

2. Open **MSYS2 UCRT64** from the Start menu and run:

```
pacman -S --needed mingw-w64-ucrt-x86_64-gcc
```

Press Enter or type `Y` when asked to confirm, and wait for installation to finish. Run this installation command in MSYS2 UCRT64.

3. Search Windows for "Edit environment variables for your account". Edit the user variable `Path` and add `C:\msys64\ucrt64\bin`. If you installed MSYS2 elsewhere, use the corresponding `ucrt64\bin` directory.

4. Save the environment variables, close all VSCode windows completely, and reopen VSCode. Select **Terminal → New Terminal**, use PowerShell, and run:

```
g++ -v
```

When GCC version and compiler information appear, continue to Hello World below. If g++ is not found, check that PATH points to the directory containing `g++.exe` and that you have restarted VSCode.

macOS: install Xcode Command Line Tools

On macOS, use the compiler provided by Xcode Command Line Tools. Run this command in the VSCode integrated terminal:

```
xcode-select --install
```

Follow the installation dialog and wait for installation to finish. If the tools are already installed, continue to the next step. You do not need the full Xcode application.

Open a new terminal in VSCode and run:

```
g++ -v
```

Apple Clang version information is the expected result: on macOS, the g++ command invokes Apple Clang. It can compile the course programs with `g++ -std=c++17` as shown in this guide; installing GNU GCC separately is unnecessary.

Linux: check first and install if needed

Some Linux environments already have g++, but it is not included in every distribution by default. First run this command in the VSCode integrated terminal:

```
g++ -v
```

If compiler version information appears, go directly to Section 2. If the command is not found, choose the installation commands for your distribution.

Ubuntu / Debian:

```
sudo apt update
sudo apt install build-essential
```

Fedora:

```
sudo dnf install gcc-c++
```

Wait for installation to finish, then run `g++ -v` again in the VSCode terminal and confirm that GCC version information appears.

2. Compile and run Hello World in the VSCode terminal

After installing the compiler, complete the checks in this section. Run all version checks, compilation commands, and execution commands in the **VSCode integrated terminal**. This section uses manual compilation and execution; it does not require `tasks.json`, `launch.json`, or Code Runner.

1. Open a practice folder and check the compiler

Create a local practice folder, such as `cpp-hello`, open it in VSCode through **File → Open Folder**, open a new terminal, and run:

```
g++ -v
```

Confirm that compiler version information appears and that there is no "command not found" error.

2. Create and save hello.cpp

Create `hello.cpp` in the folder you opened, enter the following code, and save it. You can also use the accompanying hello.cpp:

```
#include <iostream>

int main() {
    std::cout << "Hello World\n";
    return 0;
}
```

Make sure the filename is `hello.cpp`, not `hello.cpp.txt`, and that the terminal's current directory is the folder containing this file. Save any changes before compiling.

3. Compile and run

Windows (PowerShell): Run these two lines separately. Run the second line only after the first succeeds.

```
g++ -std=c++17 hello.cpp -o hello.exe
.\hello.exe
```

macOS / Linux: Run these two lines separately. Run the second line only after the first succeeds.

```
g++ -std=c++17 hello.cpp -o hello
./hello
```

Successful compilation usually produces no output. Running the program should display the following in the same terminal:

```
Hello World
```

If the compiler cannot find `hello.cpp`, check your current directory with `Get-Location` and `dir` on Windows, or `pwd` and `ls` on macOS/Linux. Change to the folder containing the source file and compile again.

4. Completion checklist

☐ Run `g++ -v` in the VSCode integrated terminal and see compiler version information.

☐ Successfully compile `hello.cpp` with `g++ -std=c++17` in the same terminal.

☐ Run the generated program and see `Hello World` in the terminal.

Local C++ setup: CLion (Windows / macOS)

Each student applies for JetBrains Student Pack with their own university email, then activates CLion with the JetBrains Account that receives the license. Setup is complete when a Hello World program runs locally.

1. Apply with your university email

Step 1: prepare your university email

Make sure you can sign into your university mailbox and receive messages. Enter the complete email address in the application, not just your student number. You will need the mailbox for verification and license notifications.

Step 2: open Student Pack and select Apply

Open the JetBrains Student Pack application page and click Apply or Apply now to begin.

If the page first requests a JetBrains Account login, sign into an existing account or register as instructed, preferably with your university email. Then continue the student application. Creating an account alone does not grant a student license.

Step 3: complete the student application

Follow the page instructions, select student status, use your university email for the application or eligibility verification, and provide the requested name, institution, and other details accurately. Check the email spelling before submitting. Supply any additional enrollment information requested by the page.

Step 4: follow the university-email messages

Open your university mailbox after submitting. Find the JetBrains verification email and follow its confirmation link. If no message appears, check spam and confirm the address entered in the application.

Continue according to the email and page instructions. If the application remains under review after email verification, wait for the decision. A submitted application is not yet an active license.

Step 5: confirm the license is linked to your account

After approval or activation instructions arrive, follow the email link and sign into JetBrains Account to complete license association. Confirm that the account has a student education license usable for CLion.

Remember this account: use the same one in CLion. If you applied through an existing account, the license may belong to that account; do not switch accounts when activating the IDE.

2. Download and install CLion

Open the official CLion download page and select the installer for your computer.

Windows: Download and run the Windows installer, follow the installation wizard, then launch CLion.

macOS: Check the processor in About This Mac from the Apple menu. Choose Apple Silicon for an M-series chip or Intel for an Intel processor. Open the downloaded disk image, drag CLion into Applications, and launch it from there.

3. Activate the student license in CLion

1. Select JetBrains Account login when the activation window appears on first launch.

2. Sign into the account that received the student license in Section 1. Complete the browser login and return to CLion.

3. Select the available student education license and activate it as instructed.

4. From the welcome screen, use Manage Licenses to reopen activation. With a project open, look under Help > Manage Subscriptions. Menu labels may differ between versions.

If no license is available, check the account, confirm application approval, and refresh the license list. Complete the application and association rather than repeatedly reinstalling the IDE.

4. Prepare the compiler

Windows: Use bundled MinGW. If prompted to select a toolchain, choose MinGW, then Bundled MinGW, and wait for compiler detection. If no prompt appears, search settings for Toolchains.

macOS: Open the system Terminal, run this command, and finish the Command Line Tools installation dialog. Wait for completion before returning to CLion. Skip installation if the tools are already installed.

```
xcode-select --install
```

CLion then uses the automatically detected Apple Clang compiler.

5. Create and run Hello World

1. Click New Project on the welcome screen.

2. Select C++ Executable and C++17, choose a local folder, and create the project.

3. Wait for the project to load. Open the generated main.cpp and replace its contents with:

```
#include <iostream>

int main() {
    std::cout << "Hello World\n";
    return 0;
}
```

You can also copy the code from the accompanying hello.cpp.

Click the green run icon beside main, or select your project and click Run at the top right. The first run compiles the program; wait for it to finish.

When the Run window shows this line, local setup is complete:

```
Hello World
```

This program requires no input.

References: Student Pack application, CLion activation, Windows compiler setup, macOS developer tools.